TD

ISTRUZIONI · VERSIONE 2.22.10

# TD Booking Suite Cloud

Installare TD Booking Suite con Docker su un server Linux dietro NPMplus, collegare i siti WordPress, fare i backup, aggiornare e collegare il registratore di cassa.

**Tastiere Digitali**
tastieredigitali.tech

10 novembre 2026

# Indice

Questa guida installa TD Booking Suite su un server nuovo, dietro NPMplus. Ogni passo dice cosa scrivere e cosa deve comparire. Se compare altro, fermati e guarda **Se qualcosa non va**, in fondo.


| | |
|---|---|
| | Come funziona |
| | Cosa serve |
| 1 | Copia il pacchetto sul server |
| 2 | Crea la configurazione |
| 3 | Costruisci l'applicazione |
| 4 | Prepara il database |
| 5 | Avvia |
| 6 | Collega NPMplus |
| 7 | Primo amministratore |
| 8 | Prima attività e sito WordPress |
| 9 | Backup ogni notte |
| 10 | Pulizia dei dati, da sola |
| 11 | Aggiornare a una versione nuova |
| 12 | Cambiare edizione: da Cloud a Private e ritorno |
| 13 | Aggiornamento automatico ogni notte |
| 14 | Registratore di cassa e ponte cassa |
| | Se qualcosa non va |

## Come funziona

Sul server girano tre container: `db` (il database), `web` (l'applicazione) e `worker` (invia le email, i promemoria, gli avvisi della lista d'attesa e fa la pulizia dei dati ogni giorno). L'applicazione risponde sulla **porta 8085** del server. NPMplus riceve il dominio in HTTPS e lo inoltra a quella porta.

## Cosa serve

- un server Linux con Docker e `docker compose`. Su Proxmox va bene un container LXC con *nesting* e *keyctl* attivi
- `python3` sul server: `apt install -y python3 python3-cryptography`
- un sottodominio, per esempio `prenota.tastieredigitali.it`, con un record A verso l'IP pubblico di NPMplus
- NPMplus che raggiunga il server sulla porta 8085

# 1 Copia il pacchetto sul server

Dal tuo computer:

```
scp TD-Booking-Suite-2.22.10-sorgenti.zip root@IP-DEL-SERVER:/opt/
```

Sul server:

```
 cd /opt
unzip -q TD-Booking-Suite-2.22.10-sorgenti.zip
cd /opt/td-booking-suite
cat VERSION
```

**Deve comparire:** `2.22.10`

# 2 Crea la configurazione

Metti il tuo dominio dopo `--origin`:

```
python3 scripts/configure.py --edition cloud --origin https://prenota.tastieredigitali.it
```

**Deve comparire:** `Configurazione creata.`

Il comando crea il file `.env`. Usa `--edition private` per l'edizione Private, che gestisce una sola attività. Si può cambiare anche dopo: vedi il punto 12.

**Importante.** `.env` contiene le chiavi che cifrano i dati dei clienti. Se lo perdi, quei dati non si leggono più, nemmeno dal backup. Copialo subito nel gestore di password:

```
cat .env
```

L'SMTP dell'agenzia, per inviti e conferme, si imposta più comodamente dal portale, in **Impostazioni generali** (passo 8). In alternativa puoi metterlo in fondo a `.env`:

```
TRANSACTIONAL_SMTP_HOST=smtp.fornitore.it
TRANSACTIONAL_SMTP_PORT=587
TRANSACTIONAL_SMTP_USER=utente
TRANSACTIONAL_SMTP_PASSWORD=password
TRANSACTIONAL_FROM=prenotazioni@tastieredigitali.it
```

## 3 Costruisci l'applicazione

```
docker compose build
```

Ci vuole qualche minuto. **Deve comparire**, alla fine, una riga `Built` per `tdbooking-web`, `tdbooking-worker` e `tdbooking-migrate`.

Se si blocca su `pip install` con "Temporary failure in name resolution", Docker non riesce a risolvere i nomi. Dagli dei DNS e riprova:

```
echo '{ "dns": ["1.1.1.1", "8.8.8.8"] }' > /etc/docker/daemon.json
systemctl restart docker
docker compose build
```

Se `/etc/docker/daemon.json` esiste già, non sovrascriverlo: aggiungi solo la riga `"dns"`. Riavviare Docker ferma per qualche secondo tutti i container del server.

## 4 Prepara il database

```
docker compose up -d db
docker compose --profile tools run --rm migrate
```

**Deve comparire**, alla fine: `Permessi runtime configurati.`

## 5 Avvia

```
docker compose up -d
docker compose ps
```

**Deve comparire:** `db`, `web` e `worker` nello stato `Up`; `db` anche `(healthy)`.

Prova dal server:

```
curl -sI http://127.0.0.1:8085/login/ -H "Host: prenota.tastieredigitali.it" | head -1
```

**Deve comparire:** `HTTP/1.1 301 Moved Permanently`. È giusto: l'applicazione chiede HTTPS, e l'HTTPS lo mette NPMplus.

# 6 Collega NPMplus

In NPMplus crea un nuovo *Proxy Host*:

| Campo | Valore |
|---|---|
| **Domain Names** | `prenota.tastieredigitali.it` |
| **Scheme** | `http` |
| **Forward Hostname / IP** | l'IP del server del booking: lo dà `hostname -I` sul server. Vale anche se NPMplus gira sullo stesso server |
| **Forward Port** | `8085` |
| **SSL** | nuovo certificato Let's Encrypt, con *Force SSL* e *HTTP/2* attivi |
| **HSTS** | spento: lo manda già l'applicazione |

Apri `https://prenota.tastieredigitali.it/login/`. **Deve comparire** la pagina di accesso, con il lucchetto.

**Sicurezza.** La porta 8085 deve rispondere solo a NPMplus: chi la raggiunge direttamente salta WAF e HTTPS. Se il booking è su un LXC separato, dal firewall di Proxmox (container › *Firewall*) apri la 8085 solo all'IP di NPMplus.

# 7 Primo amministratore

```
docker compose run --rm web python manage.py bootstrap --email admin@tastieredigitali.it
```

Ti chiede la password due volte. Poi accedi dal browser: entri subito con la password.

Finché non aggiungi un secondo fattore, in alto vedi l'avviso **Proteggi il tuo account**. Per l'account dell'agenzia attivalo appena puoi, da *Account e sicurezza*:

1. Google Authenticator o un'altra app simile, oppure una passkey
2. salva i 10 codici di recupero nel gestore di password: servono se perdi il telefono

# 8 Prima attività e sito WordPress

Nel portale, come amministratore dell'agenzia. Per tutto il resto c'è la **Guida** nel menu del portale.

1. **Impostazioni generali**, una volta sola: nome, logo per sfondi chiari e logo per sfondi scuri, icona, colore, dati dell'azienda per la pagina privacy, testi della pagina di accesso, contatti di assistenza, email che riceve le **richieste di prova**, **invio email dell'agenzia** (SMTP, Google / Gmail o Microsoft 365, con il nome del mittente e l'email di prova per controllarlo; per Google l'indirizzo di ritorno da autorizzare è `https://IL-TUO-PORTALE/oauth/google/`) e i valori con cui parte ogni nuova attività
2. **Clienti › Nuovo cliente**: nome, indirizzo breve (per esempio `pizzeria-da-mario`), email dell'amministratore dell'attività, tipo di attività e, se vuoi, il referente
3. **Accesso dell'amministratore dell'attività**: lascia vuota la password e copia il link di invito che compare, poi mandalo tu (valido 24 ore). Oppure apri "Oppure imposta tu la password" e scegline una da comunicargli
4. Nella scheda del cliente completa anagrafica, logo e referenti, poi apri **Configura l'attività**: in **Tipo di attività e sedi** scegli il tipo e quante sedi ha e premi *Applica la configurazione*. Servizi, risorse, orari ed eventuali date sono pronti, da ritoccare. Si può rifare quando vuoi, le prenotazioni restano
5. Sempre lì, **Sito WordPress › Genera il codice di collegamento**: il codice vale 24 ore e una volta sola, e serve al plugin (qui sotto)
6. **Invio email** dell'attività: solo se vuole il suo (SMTP, Gmail o Microsoft 365), soprattutto per le conferme delle prenotazioni e, con il marketing attivo, per le campagne. Altrimenti vale quello dell'agenzia

**Pagine pubbliche Prezzi e versioni e Perché sceglierci** (Area agenzia › Prezzi e versioni): restano spente finché non le accendi. Accendile solo sul portale che usi come vetrina; sull'installazione che serve i clienti lasciale spente. La pagina **Privacy** c'è sempre, su `/privacy/`.

L'amministratore dell'attività, entrando, trova **Configura l'attività**: sedi, orari, servizi e risorse in ordine, con quello che manca.

In WordPress:

1. *Plugin › Aggiungi nuovo › Carica plugin*: `td-booking-suite-2.17.0-wordpress.zip`, poi attiva. Nell'edizione Cloud lo zip si scarica anche dal portale, in **Area agenzia › Plugin WordPress**, con i valori da inserire per ogni attività
2. *Impostazioni › TD Booking Suite › Collegati al portale*: indirizzo del portale, per esempio `https://prenota.tastieredigitali.it`, e il codice di collegamento. Il plugin autorizza il sito, legge sedi e servizi e crea la pagina **Prenota** con `[td_booking_suite]`. Nelle stesse impostazioni trovi gli shortcode per una sola sede o un solo servizio
3. In alternativa, le *Impostazioni manuali*: origine del portale e indirizzo breve dell'attività, con il sito autorizzato in *Impostazioni e sito*

Prova finale:

- [ ] una prenotazione dal sito arriva nel portale, e l'email di conferma arriva al cliente
- [ ] il link "annulla" dell'email apre la conferma e annulla
- [ ] dal telefono in 4G la prenotazione funziona come dal Wi-Fi
- [ ] nei log del WAF non ci sono blocchi sulle pagine `/book/`

# 9 Backup ogni notte

Una volta sola, crea la chiave dei backup e copiala nel gestore di password, separata da `.env`. Se l'hai già creata col passo 4 del README, salta la seconda riga:

```
mkdir -p /var/backups/tdbs && chmod 700 /var/backups/tdbs
python3 /opt/td-booking-suite/scripts/backup_crypto.py keygen /root/tdbs-backup.key
```

Crea il file `/usr/local/bin/tdbs-backup` con questo contenuto:

```
#!/bin/sh
set -eu
cd /opt/td-booking-suite
F=/var/backups/tdbs/tdbooking-$(date +%F)
docker compose exec -T db pg_dump -U postgres -Fc tdbooking > "$F.dump"
python3 scripts/backup_crypto.py encrypt "$F.dump" "$F.dump.enc" --key-file /root/tdbs-
backup.key
rm "$F.dump"
find /var/backups/tdbs -name '*.enc' -mtime +30 -delete
```

Poi rendilo eseguibile e fallo partire ogni notte alle 2:30:

```
chmod 700 /usr/local/bin/tdbs-backup
echo '30 2 * * * root /usr/local/bin/tdbs-backup' > /etc/cron.d/tdbs-backup
```

I file `.enc` vanno copiati anche fuori dal server, insieme al backup Proxmox del container.

**Ripristino**, da provare una volta subito dopo l'installazione. Al posto di `FILE.dump.enc` metti il backup da ripristinare:

```
cd /opt/td-booking-suite
python3 scripts/backup_crypto.py decrypt FILE.dump.enc /root/ripristino.dump --key-file
/root/tdbs-backup.key
docker compose stop web worker
docker compose exec -T db pg_restore -U postgres -d tdbooking --clean --if-exists <
/root/ripristino.dump
docker compose --profile tools run --rm migrate
docker compose up -d
rm /root/ripristino.dump
```

Il ripristino funziona solo con lo stesso `.env` del momento del backup.

# 10 Pulizia dei dati, da sola

Dalla 2.14.0 non serve un cron: il `worker` cancella i dati scaduti secondo le regole di conservazione di ogni attività una volta al giorno (prenotazioni vecchie senza dati personali, schede senza prenotazioni recenti né consenso, voci della scheda trattamenti scadute, richieste della lista d'attesa passate). L'esito è nella dashboard dell'agenzia, in *Da controllare*: compare se la pulizia non è riuscita (il dettaglio è nel log del `worker`) o se è ferma da più di due giorni, e sotto c'è quando è stata fatta l'ultima volta.

Per vedere cosa farebbe, senza cancellare niente, o per forzarla subito su un'attività:

```
 docker compose run --rm web python manage.py retention --tenant pizzeria-da-mario
docker compose run --rm web python manage.py retention --tenant pizzeria-da-mario --apply
```

Un cron rimasto dalle versioni di prima non fa danni (la pulizia rifatta non cambia niente), ma si può togliere.

## 11 Aggiornare a una versione nuova

Con l'aggiornamento automatico del passo 13 e `TDBS_AGGIORNA_CODICE=1` in `.env` (le installazioni nuove ce l'hanno già) la versione nuova si installa da sola, di notte, appena è pubblicata su GitHub: non devi fare niente, nell'email del mattino trovi da quale a quale versione e il link alle novità. Puoi sempre installarla a mano, per esempio subito o con l'aggiornamento automatico spento. Leggi prima il registro modifiche della versione. Poi, con lo zip nuovo in `/opt`:

```
 /usr/local/bin/tdbs-backup
cd /opt
unzip -q TD-Booking-Suite-2.X.Y-sorgenti.zip -d /opt/nuova
cp -r /opt/nuova/td-booking-suite/. /opt/td-booking-suite/
rm -rf /opt/nuova
cd /opt/td-booking-suite
docker compose build
docker compose --profile tools run --rm migrate
docker compose up -d --remove-orphans
```

`.env` resta com'è: lo zip non lo contiene. Aggiornando alla 2.14.0, `migrate` crea anche gli indici di ricerca delle schede clienti che ci sono già: qualche secondo ogni 10.000 clienti. Valori facoltativi da aggiungere quando servono, poi `docker compose up -d`: `TDBS_SESSION_REMEMBER_DAYS=30`, i giorni per cui un dispositivo con *Resta collegato* rimane dentro dall'ultima visita (predefinito 0: finché non preme *Esci*). Per tornare indietro, ripeti gli stessi comandi con lo zip della versione precedente e ripristina il backup fatto all'inizio.

## 12 Cambiare edizione: da Cloud a Private e ritorno

L'edizione è scritta in una riga del file `.env`: `TDBS_EDITION=cloud` oppure `TDBS_EDITION=private`. Dati, chiavi, utenti e prenotazioni restano dove sono: cambia solo quello che il portale mostra e permette.

| | Cloud | Private |
|---|---|---|
| **Attività** | quante vuoi, ognuna con il suo portale | una sola |
| **Nome in alto nel menu** | il nome del portale | il nome dell'attività |
| **Stato, piano e prova dei clienti** | sì | no: l'attività è sempre attiva |
| **Richieste di prova dalla pagina di accesso** | sì | no |

| | Cloud | Private |
|---|---|---|
| **Plugin WordPress scaricabile dal portale** | sì | no: si installa dallo zip del pacchetto |
| **Tipo di attività, sedi e codice per WordPress** | li imposta l'agenzia | li imposta l'amministratore dell'attività |

**Da Cloud a Private**, per esempio per un'installazione dedicata a un solo cliente. Il portale deve contenere **una sola attività**: se ne contiene di più, all'avvio si ferma e lo dice, senza toccare i dati.

```
 /usr/local/bin/tdbs-backup
cd /opt/td-booking-suite
sed -i 's/^TDBS_EDITION=.*/TDBS_EDITION=private/' .env
grep TDBS_EDITION .env
docker compose up -d
```

**Deve comparire:** `TDBS_EDITION=private`, poi i container ricreati. Nel menu del portale c'è scritto *Edizione Private* e il nome dell'attività.

**Da Private a Cloud**, per esempio quando sullo stesso server vuoi ospitare altri clienti:

```
 /usr/local/bin/tdbs-backup
cd /opt/td-booking-suite
sed -i 's/^TDBS_EDITION=.*/TDBS_EDITION=cloud/' .env
grep TDBS_EDITION .env
docker compose up -d
```

**Deve comparire:** `TDBS_EDITION=cloud`. L'attività esistente diventa il primo cliente dell'Area agenzia, con lo stato che aveva prima, di solito *Attivo*: controllalo nella sua scheda, *Impostazioni e sito*.

Non serve ricostruire l'immagine né lanciare `migrate`: basta `docker compose up -d`, che ricrea web e worker con la riga nuova.

# 13 Aggiornamento automatico ogni notte

Ogni notte, da solo, aggiorna quello su cui gira il portale: l'immagine di Python 3.12 con i pacchetti Debian, tutte le librerie Python all'ultima versione (tranne i limiti scritti in `requirements.in`, per esempio Django resta sulla 5.2) e PostgreSQL all'ultima 17.x. Prima di toccare qualcosa fa il backup cifrato del database. Dopo prova la versione nuova: pagina di accesso e `manage.py selftest` dentro il container, cioè database, cifratura, password, loghi, app Authenticator, passkey e pagine. Se qualcosa non va torna da solo alla versione di prima, database compreso. Alla fine manda un'email con il riepilogo. Il portale resta fermo da qualche secondo a un minuto, solo quando c'è qualcosa da aggiornare.

Con `TDBS_AGGIORNA_CODICE=1` in `.env` aggiorna anche **TD Booking Suite** all'ultima versione pubblicata su GitHub (release). Non torna mai a una versione più vecchia e non installa le versioni di prova. Scarica lo zip dei sorgenti, controlla che contenga la cartella `td-booking-suite/` con la versione giusta e lo mette al posto del codice di adesso. `.env`, `logs/`, `backups/` e i file qui sotto restano dove sono. Il codice di prima resta in `.agg-`

`codice-prima`: se la versione nuova non passa la prova, torna quello, insieme all'immagine e al database di prima. Le installazioni nuove hanno già la riga in `.env`. Su quelle fatte prima della 2.14.0 aggiungila tu:

```
echo TDBS_AGGIORNA_CODICE=1 >> /opt/td-booking-suite/.env
```

Senza la riga, o con `bash scripts/aggiorna-tutto.sh --solo-librerie`, il codice non lo tocca: la versione nuova la installi tu (passo 11). Le librerie aggiornate stanno in file a parte (qui sotto), quindi una versione nuova non le fa tornare indietro.

Cosa serve:

- la chiave dei backup del passo 9, `/root/tdbs-backup.key`: senza, l'aggiornamento non tocca niente
- l'invio email dell'agenzia (passo 8) e, in **Impostazioni generali**, l'email che riceve le richieste di prova o quella di assistenza. Per un altro indirizzo metti `TDBS_AGG_EMAIL=indirizzo` in `.env`

Prima prova a vuoto, che dice cosa farebbe senza toccare niente:

```
cd /opt/td-booking-suite
bash scripts/aggiorna-tutto.sh --dry-run
```

**Deve comparire**, alla fine, `— fine —`. Poi pianificalo:

```
bash scripts/installa-aggiornamento.sh
```

**Deve comparire:** ✅ `Timer systemd installato: ogni notte alle 04:00`. Con un numero cambia l'ora, per esempio `bash scripts/installa-aggiornamento.sh 3`. Il timer è *Persistent*: se alle 4 il server era spento, l'aggiornamento parte appena si riaccende. Poi:

| Per | Comando |
|---|---|
| **vedere se è attivo e com'è andata l'ultima volta** | `bash scripts/installa-aggiornamento.sh --status` |
| **leggere il log** | `tail -f /opt/td-booking-suite/logs/aggiorna-tutto.log` |
| **aggiornare subito** | `bash scripts/aggiorna-tutto.sh` |
| **togliere la pianificazione** | `bash scripts/installa-aggiornamento.sh --remove` |

Il log non supera 1 MB: quando ci arriva restano le righe più recenti. I backup fatti prima di ogni aggiornamento (`pre-aggiornamento-….dump.enc` in `/var/backups/tdbs`) restano 30 giorni, poi li cancella lo script stesso.

Le immagini e le librerie aggiornate finiscono in `compose.override.yaml` e `requirements.auto.lock`. Li scrive lo script, stanno fuori da git e `docker compose` li legge da solo. Per tornare a quelle fissate nel rilascio:

```
rm compose.override.yaml requirements.auto.lock
docker compose build
docker compose up -d
```

Valori facoltativi in `.env`: `TDBS_AGGIORNA_CODICE=1` (anche le versioni nuove del codice), `TDBS_AGG_EMAIL` (a chi va il riepilogo), `TDBS_AGG_MAIL_SEMPRE=1` (email anche quando era già tutto aggiornato), `TDBS_AGG_TIENI_ROLLBACK=3` (quante immagini di prima tenere), `TDBS_AGG_CHIAVE` e `TDBS_AGG_BACKUP` (chiave e cartella dei backup, se non sono quelle del passo 9).

Il passaggio a una versione maggiore di PostgreSQL, per esempio dalla 17 alla 18, non lo fa mai da solo: si fa a mano, con backup e ripristino.

# 14 Registratore di cassa e ponte cassa

Il registratore telematico (RCH Print!F, Epson FP-81II / FP-90III RT) sta nella rete del negozio, non in quella del server: per questo nell'edizione Cloud e Docker il collegamento passa dal **ponte cassa**, `scripts/ponte_cassa.py`, che gira su un qualunque computer del negozio (Windows, Mac o Linux) con Python 3.9 o più nuovo.

1. In *Registratore di cassa* (menu *Gestione tecnica* dell'attività) scegli per la sede il registratore, *Rete* con l'indirizzo IP della cassa (RCH: menu SRV, Connettività, Ethernet, con il web server acceso) oppure *USB / seriale* con la porta COM, la matricola, i reparti e i pagamenti. Salva.
2. Premi *Crea il codice del ponte* e copia il codice ( `tdcassa_…` ): non viene più mostrato. *Scarica ponte_cassa.py* sul computer del negozio.
3. Avvialo, e lascia aperta la finestra (o mettilo tra i programmi che partono con il computer):

```
python ponte_cassa.py https://prenotazioni.example.it/book/ATTIVITA/api/cassa/
tdcassa_IL-CODICE
```

1. In *Registratore di cassa* premi *Prova collegamento*: dopo qualche secondo, ricaricando la pagina, vedi se la cassa risponde e l'ultimo contatto del ponte.

Il ponte chiama il programma ogni 2 secondi in HTTPS con il codice della sede (al massimo 120 volte al minuto) e parla con la cassa solo su indirizzi della rete locale. Se il codice finisce in mano sbagliata, *Revoca* o *Nuovo codice* lo fermano.

**Cassa via USB**: sulle casse RCH l'USB compare in Windows come porta COM (Gestione dispositivi, *Porte (COM e LPT)*). Il computer del ponte ha bisogno di `pip install pyserial`; `python ponte_cassa.py --porte` elenca le porte. Il protocollo USB *RCH con conferma* aspetta dalla cassa la conferma di ogni comando; *Comandi semplici* non ha conferma, e ogni scontrino resta da verificare a mano. Le casse Epson qui si collegano solo in rete.

## Se qualcosa non va

Per leggere gli errori: `docker compose logs --tail 50 web worker db`

| Cosa vedi | Cosa fare |
|---|---|
| `web` e `worker` si riavviano, nel log "Il database non è aggiornato a questa versione" | Manca `migrate` dopo un aggiornamento: `docker compose --profile tools run --rm migrate`, poi `docker compose up -d` |

| Cosa vedi | Cosa fare |
| --- | --- |
| `migrate` **risponde "No migrations to apply" ma le pagine danno errore 500** | Succedeva fino alla 2.3.0: `migrate` era rimasto all'immagine vecchia. `docker compose --profile tools build migrate`, poi di nuovo `migrate` |
| **"required variable … is missing a value"** | `.env` manca o è incompleto. Rilancia il comando del passo 2: se trova un `.env` vuoto o incompleto ti dice cosa manca |
| **La build si blocca con "Temporary failure in name resolution"** | DNS di Docker: passo 3 |
| `db` **resta *unhealthy*, o nel suo log c'è "Permission denied" su** `10-tdbooking.sh` | Permessi dei file. Dalla cartella del progetto: `find . -type d -exec chmod 755 {} +`, poi `find . -type f ! -name .env -exec chmod 644 {} +`, poi `chmod 755 deploy/init-db.sh`. Poi rilancia il passo 4 |
| **NPMplus risponde 502** | `web` non è avviato (`docker compose ps`), oppure NPMplus non raggiunge la porta 8085: controlla IP e firewall |
| **La porta 8085 è già occupata** | Aggiungi `TDBS_PORT=8090` a `.env`, poi `docker compose up -d`, e in NPMplus metti la porta 8090 |
| **400 Bad Request** | Il dominio aperto è diverso da `PUBLIC_ORIGIN` in `.env` |
| **Troppi reindirizzamenti** | Tra NPMplus e l'applicazione c'è un altro proxy che non passa `X-Forwarded-Proto`. NPMplus da solo lo passa sempre |
| **403 "Proxy non configurato"** | La richiesta non passa da NPMplus: stai aprendo la porta 8085 direttamente? |
| **Email ferme "in attesa"** | SMTP mancante o sbagliato: controlla *Impostazioni generali › Invio email dell'agenzia*, oppure *Invio email* dell'attività |
| **Dopo il cambio a Private il portale non parte e i log dicono "gestisce una sola attività"** | Il database ha più attività. Rimetti `TDBS_EDITION=cloud` (punto 12) e `docker compose up -d`: i dati non sono stati toccati |
| **La pagina chiede il codice o la password prima di aprirsi** | È la conferma per le pagine sensibili (accessi e impostazioni), al massimo ogni 15 minuti: codice dell'app o passkey, oppure la password per chi non ha ancora un secondo fattore |
| `web` **si riavvia di continuo con "ruolo runtime non deve essere superuser"** | In `.env`, `DATABASE_URL` deve usare `tdbooking_app`, come lo crea il passo 2 |
| **Cambiata** `APP_DB_PASSWORD` **in** `.env` | Rilancia `docker compose --profile tools run --rm migrate`, poi `docker compose up -d` |
| **L'amministratore ha perso telefono e codici di recupero** | `docker compose run --rm web python manage.py recover_admin --email admin@tastieredigitali.it --confirm-reset-mfa`: azzera password e secondo fattore e stampa un nuovo link di invito |